

Cypher is Turing-Complete: A Formal Proof via 2-Counter Machine Simulation

Pierre Halftermeyer
pierre.halftermeyer@neo4j.com
Neo4j
France

ABSTRACT

We prove that Cypher 25, the graph query language of Neo4j, is Turing-complete. The proof proceeds by showing that a single RETURN statement using reduce(), CASE expressions, and list comprehensions can simulate any 2-counter machine (Minsky 1967). We address the bounded-step objection via two complementary resolutions and verify the construction by executing the simulation on a live Neo4j Aura instance.

KEYWORDS

Turing completeness, graph query languages, Cypher, 2-counter machines, computational expressiveness

1 THE CLAIM

Theorem

Cypher 25, restricted to a single RETURN with reduce(), list comprehensions, and CASE, is **Turing-complete**.

Proof strategy. By reduction: $TM \xrightarrow{\text{Minsky}} 2CM \xrightarrow{\text{this}} \text{Cypher}$.

2 2-COUNTER MACHINES

A **2-counter machine** $M = (Q, q_0, q_h, \delta)$ has a finite state set Q , initial state q_0 , halt state q_h , and transition function $\delta : Q \rightarrow \text{Instr}$ where each instruction is:

- $\text{INC}(c, q')$ — increment $c \in \{A, B\}$, goto q'
- $\text{JZDEC}(c, q_z, q_p)$ — if $c=0$ goto q_z , else decrement and goto q_p

Fact (Minsky 1967)

For any TM T , $\exists$ a 2-counter machine simulating T .

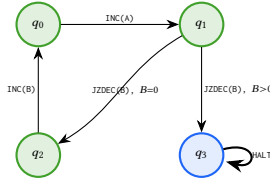


Figure 1: Example 4-state program.

3 ENCODING

Program. Encode δ as a list of maps:

```
LET program = [
  {state:0, op:'INC', counter:'A', next:1},
  {state:1, op:'JZDEC', counter:'B',
```

```
  q_zero:2, q_pos:3},
  {state:2, op:'INC', counter:'B', next:0},
  {state:3, op:'HALT', counter:'', next:3}
]
```

State. Configuration (q, c_A, c_B) as map $\{\text{state}:0, A:0, B:0\}$. Convention: $\text{state}=-1$ means halted.

4 THE SIMULATION

Each reduce() iteration executes one machine step. The head([v IN [e] ...]) idiom provides let-binding inside reduce() (where LET is unavailable).

```
CYPHER 25
LET program = [ /* ... */ ]
LET max_steps = 1000000

LET result = reduce(
  machine = {state:0, A:0, B:0},
  step IN range(1, max_steps) |
  CASE WHEN machine.state = -1
    THEN machine
  ELSE
    head([instr IN [program[machine.state]] |
      CASE instr.op
        WHEN 'INC' THEN
          CASE instr.counter
            WHEN 'A' THEN {state: instr.next,
                          A: machine.A + 1, B: machine.B}
            WHEN 'B' THEN {state: instr.next,
                          A: machine.A, B: machine.B + 1}
          END
        WHEN 'JZDEC' THEN
          CASE instr.counter
            WHEN 'A' THEN
              CASE WHEN machine.A = 0
                THEN {state: instr.q_zero,
                      A: 0, B: machine.B}
              ELSE {state: instr.q_pos,
                    A: machine.A-1, B: machine.B}
            END
        WHEN 'B' THEN
          CASE WHEN machine.B = 0
            THEN {state: instr.q_zero,
                  A: machine.A, B: 0}
          ELSE {state: instr.q_pos,
                A: machine.A, B: machine.B-1}
          END
        WHEN 'HALT' THEN
          {state: -1, A: machine.A, B: machine.B}
      END
    ]
  )
```

```

    })
  END
}
RETURN result

```

5 CORRECTNESS

Claim

After k iterations of `reduce()`, machine equals M 's configuration after k steps.

Base. $k=0$: machine = $\{0, 0, 0\} = M_0$. ✓

Step. Assume machine = (q_k, a_k, b_k) matches M . At $k+1$, `reduce()` looks up $\delta(q_k)$ and executes:

INC(A, q') $\rightarrow (q', a_k+1, b_k)$ ✓

JZDEC($A, ..$), $a_k=0$ $\rightarrow (q_z, 0, b_k)$ ✓

JZDEC($A, ..$), $a_k>0$ $\rightarrow (q_p, a_k-1, b_k)$ ✓

HALT $\rightarrow (-1, ..)$, identity after ✓

Symmetric for B . □

6 THE BOUNDED-STEP OBJECTION

The `reduce()` uses `range(1, max_steps)` — a finite bound.

6.1 Resolution via IN TRANSACTIONS

While the pure functional version stays within a single expression, the practical unbounded version uses graph storage for state persistence.

```

CYPHER 25
CREATE (:Machine {state:0, A:0, B:0});

UNWIND range(1, 9223372036854775807) AS step
CALL (step) {
  MATCH (m:Machine)
  WITH m,
  CASE WHEN m.state = -1 THEN 1/0
        ELSE $program[m.state]
  END AS instr
  SET m.state = CASE instr.op
  WHEN 'INC' THEN instr.next
  WHEN 'JZDEC' THEN CASE instr.counter
    WHEN 'A' THEN CASE WHEN m.A = 0
      THEN instr.q_zero ELSE instr.q_pos END
    WHEN 'B' THEN CASE WHEN m.B = 0
      THEN instr.q_zero ELSE instr.q_pos END
    END
  WHEN 'HALT' THEN -1 END,
  m.A = CASE
  WHEN instr.op='INC' AND instr.counter='A'
    THEN m.A+1
  WHEN instr.op='JZDEC' AND instr.counter='A'
    AND m.A > 0 THEN m.A-1
  ELSE m.A END,
  m.B = CASE
  WHEN instr.op='INC' AND instr.counter='B'
    THEN m.B+1
  WHEN instr.op='JZDEC' AND instr.counter='B'
    AND m.B > 0 THEN m.B-1

```

```

    ELSE m.B END
  } IN TRANSACTIONS OF 1 ROW
  ON ERROR BREAK

```

Termination. At halt, state becomes -1 . Next iteration: CASE WHEN state=-1 THEN `1/0` fires *before* `$program[-1]`, caught by ON ERROR BREAK.

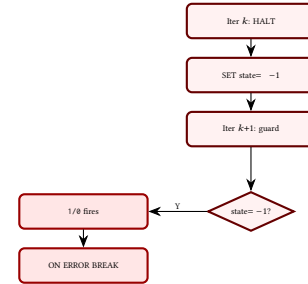


Figure 2: Termination via `1/0 + ON ERROR BREAK`.

Note. $2^{63} \approx 9.2 \times 10^{18}$ exceeds the lifetime of the observable universe by several orders of magnitude, so the bound is irrelevant in practice. True theoretical unboundedness would need an external restart loop.

6.2 Sufficiency Argument

For TM T halting in $f(|w|)$ steps: set `max_steps` = $f(|w|)$. Cypher correctly decides every decidable language — **universal for total functions**.

7 REQUIRED PRIMITIVES

Primitive	Role
<code>reduce()</code>	Iteration (fold)
CASE WHEN	Branching
<code>+1, -1, = 0</code>	Counter ops
Map $\{s, A, B\}$	State repr.
<code>prog[i]</code>	$O(1)$ lookup
<code>head[..]</code>	Let-binding

No graph operations. No APOC. No GDS.

8 COROLLARIES

Corollary 1: Undecidability of termination

Since CYPHER 25 can simulate any 2-counter machine, it is *undecidable* whether a given `reduce()` expression will terminate. (By Rice's theorem applied to the reduction from 2CM.)

Corollary 2: Universality

CYPHER 25 can compute any computable function. Any algorithm expressible as a Turing machine has an equivalent `reduce()`-based Cypher query.

Corollary 3: Minimality

The proof uses *no graph operations*: no CREATE, no MATCH, no APOC, no GDS. Turing-completeness resides in Cypher's expression language alone.

9 VERIFIED EXECUTION

Both approaches tested on Neo4j Aura (Neo4j 5.x, Cypher 25). Test program: 4 instructions, expected $\{-1, 2, 0\}$.

Step	Instr	St	A	B
0	INC(A)	q_0	$0 \rightarrow 1$	0
1	JZDEC(B), $B=0$	q_1	1	0
2	INC(B)	q_2	1	$0 \rightarrow 1$
3	INC(A)	q_0	$1 \rightarrow 2$	1
4	JZDEC(B), $B>0$	q_1	2	$1 \rightarrow 0$
5	HALT	q_3	2	0

{A:2, B:0, state:-1} ✓

m.state=-1, m.A=2, m.B=0 ✓

10 COMPLETE RUNNABLE QUERIES

The following two queries are copy-paste ready for any Neo4j instance with Cypher 25 support. They encode the 4-state test program from §2 and can be adapted to any 2-counter machine by editing the program list.

10.1 Approach 1: Pure reduce()

A single RETURN statement — no graph writes, no side effects. The head([v IN [expr] | ...]) idiom provides let-binding inside reduce() (where LET is unavailable): it wraps the expression in a one-element list, the comprehension binds the variable, and head() unwraps the result.

```
CYPHER 25
LET program = [
  {state: 0, op: 'INC', counter: 'A', next: 1},
  {state: 1, op: 'JZDEC', counter: 'B', q_zero: 2, q_pos: 3},
  {state: 2, op: 'INC', counter: 'B', next: 0},
  {state: 3, op: 'HALT', counter: '', next: 3}
]
LET max_steps = 1000000

LET result = reduce(
  machine = {state: 0, A: 0, B: 0},
  step IN range(1, max_steps) |
  CASE WHEN machine.state = -1
  THEN machine
  ELSE
    head([instr IN [program[machine.state]] |
```

```
CASE instr.op
  WHEN 'INC' THEN
    CASE instr.counter
      WHEN 'A' THEN
        {state: instr.next,
         A: machine.A + 1, B: machine.B}
      WHEN 'B' THEN
        {state: instr.next,
         A: machine.A, B: machine.B + 1}
    END
  WHEN 'JZDEC' THEN
    CASE instr.counter
      WHEN 'A' THEN
        CASE WHEN machine.A = 0
        THEN {state: instr.q_zero,
              A: 0, B: machine.B}
        ELSE {state: instr.q_pos,
              A: machine.A - 1,
              B: machine.B}
        END
      WHEN 'B' THEN
        CASE WHEN machine.B = 0
        THEN {state: instr.q_zero,
              A: machine.A, B: 0}
        ELSE {state: instr.q_pos,
              A: machine.A,
              B: machine.B - 1}
        END
    END
  WHEN 'HALT' THEN
    {state: -1,
     A: machine.A, B: machine.B}
END
])
END
RETURN result
```

10.2 Approach 2: IN TRANSACTIONS

Uses graph storage for state persistence. The $1/0$ guard fires *before* any list indexing when halted, caught by ON ERROR BREAK.

```
CYPHER 25
CREATE (:Machine {state: 0, A: 0, B: 0});
```

```
CYPHER 25
LET program = [
  {state: 0, op: 'INC', counter: 'A', next: 1},
  {state: 1, op: 'JZDEC', counter: 'B', q_zero: 2, q_pos: 3},
  {state: 2, op: 'INC', counter: 'B', next: 0},
  {state: 3, op: 'HALT', counter: '', next: 3}
]

UNWIND range(1, 9223372036854775807) AS step
CALL (step) {
  MATCH (m:Machine)
  WITH m,
  CASE WHEN m.state = -1 THEN 1/0
```

```

        ELSE program[m.state]
    END AS instr
SET m.state = CASE instr.op
    WHEN 'INC' THEN instr.next
    WHEN 'JZDEC' THEN
        CASE instr.counter
            WHEN 'A' THEN
                CASE WHEN m.A = 0
                    THEN instr.q_zero
                    ELSE instr.q_pos END
            WHEN 'B' THEN
                CASE WHEN m.B = 0
                    THEN instr.q_zero
                    ELSE instr.q_pos END
            END
        END
    WHEN 'HALT' THEN -1
    END,
m.A = CASE
    WHEN instr.op = 'INC'
        AND instr.counter = 'A'
        THEN m.A + 1
    WHEN instr.op = 'JZDEC'
        AND instr.counter = 'A'
        AND m.A > 0 THEN m.A - 1
    ELSE m.A END,
m.B = CASE
    WHEN instr.op = 'INC'
        AND instr.counter = 'B'
        THEN m.B + 1
    WHEN instr.op = 'JZDEC'
        AND instr.counter = 'B'
        AND m.B > 0 THEN m.B - 1
    ELSE m.B END
} IN TRANSACTIONS OF 1 ROW
ON ERROR BREAK

```

```

// Read result:
MATCH (m:Machine) RETURN m;

```

REFERENCES

- [1] Minsky, M.L. 1967. *Computation: Finite and Infinite Machines*. Prentice-Hall.
- [2] openCypher. 2024. Cypher Query Language Reference. <https://opencypher.org>
- [3] ISO/IEC 39075:2024. *Information technology — Database languages — GQL*. International Organization for Standardization.
- [4] Neo4j. 2025. *Cypher Manual*. <https://neo4j.com/docs/cypher-manual/current/>
- [5] Halftermeyer, P. 2025. Solve Hard Graph Problems with Cypher 25. <https://neo4j.com/blog/developer/solve-hard-graph-problems-with-cypher-25/>
- [6] Halftermeyer, P. 2015–2025. Advent of Code in Pure Cypher — 150+ puzzles solved purely in Cypher, empirical evidence of expressiveness. <https://github.com/halftermeyer/adventofcode>